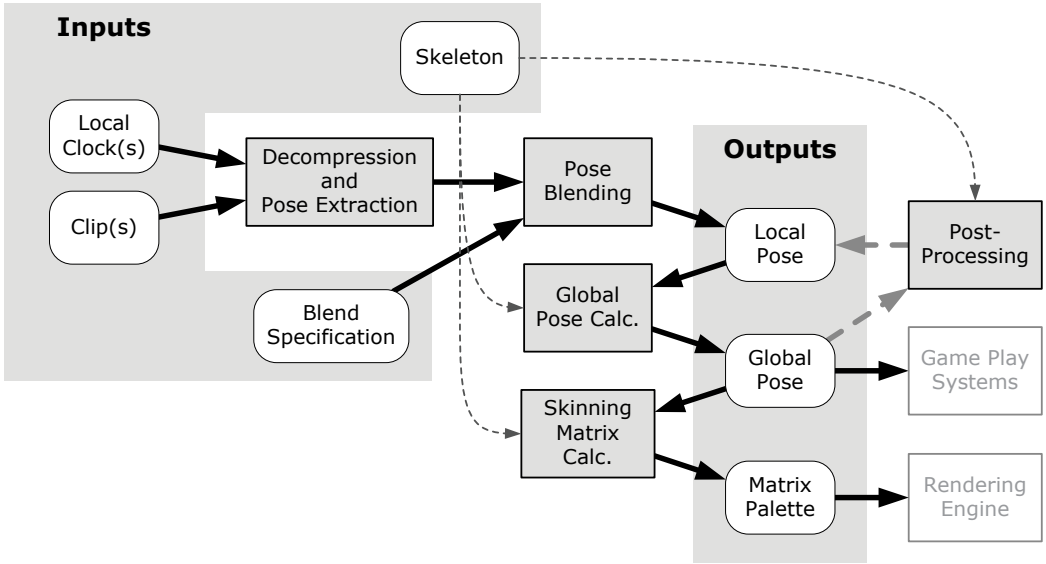

Figure 12.47. A typical animation pipeline.

12.10 Action State Machines

The actions of a game character (standing, walking, running, jumping, etc.) are usually best modeled via a finite state machine, commonly known as the *action state machine* (ASM). The ASM subsystem sits atop the animation pipeline and provides a state-driven animation interface for use by virtually all higher-level game code.

Each state in an ASM corresponds to an arbitrarily complex blend of simultaneous animation clips. Some states might be very simple—for example, the "idle" state might be comprised of a single full-body animation. Other states might be more complex. A "running" state might correspond to a semicircular blend, with strafing left, running forward and strafing right at the -90 degree, 0 degree and $+90$ degree points, respectively. The "running while shooting" state might include a semicircular directional blend, plus additive or partial-skeleton blend nodes for aiming the character's weapon up, down, left and right, and additional blends to permit the character to look around with its eyes, head and shoulders. More additive animations might be included to control the character's overall stance, gait and foot spacing while locomoting and to provide a degree of "humanness" through random movement variations.

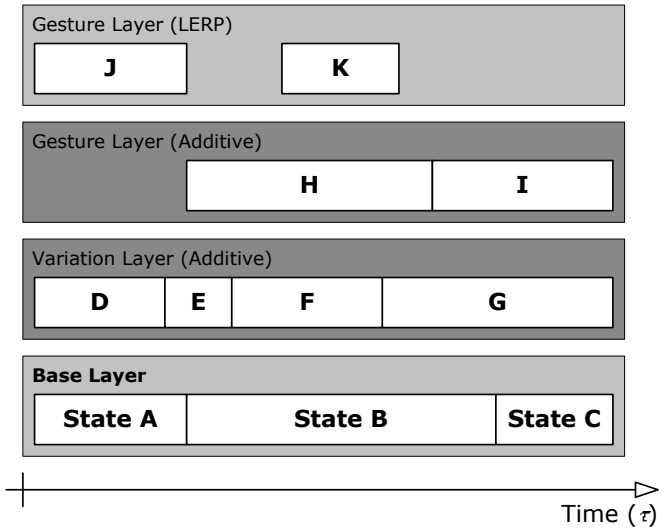

Figure 12.48. A layered action state machine, showing how each layer's state transitions are temporally independent. In this example, the base layer describes the character's full-body stance and movement. A variation layer provides variety by applying additive clips to the character's pose. Finally, two gesture layers, one additive and one partial, permit the character to aim or point at objects in the world around it.

A character's ASM also ensures that characters can *transition* smoothly from state to state. During a transition from state A to state B, the final output poses of both states are usually blended together to provide a smooth *cross-fade* between them.

Most high-quality animation engines also permit different parts of a character's body to be doing different, independent or semi-independent actions simultaneously. For instance, a character might be running, aiming and firing a weapon with its arms, and speaking a line of dialog with its facial joints. The movements of different parts of the body aren't generally in perfect sync either—certain parts of the body tend to "lead" the movements of other parts (e.g., the head leads a turn, followed by the shoulders, the hips and finally the legs). In traditional animation, this well-known technique is known as *anticipation* [51]. This kind of complex movement can be realized by allowing multiple independent state machines to control a single character. Usually each state machine exists in a separate *state layer*, as shown in Figure 12.48. The output poses from each layer's ASM are blended together into a final composite pose.

All of this means that at any given moment in time, multiple animation

clips are contributing to the final pose of a character's skeleton. For each character, then, we need a way to track all of the currently-playing clips, and to describe how exactly they should be blended together in order to produce the character's final pose. Generally speaking, there are two ways to do this:

1. *Flat weighted average*. In this approach, the engine maintains a flat list of all animation clips that are currently contributing to a character's final pose, with one blend weight per clip. The animations are blended together as one big weighted average to produce the final pose.
2. *Blend trees*. In this approach, each contributing clip is represented by the leaf nodes of a tree. The interior nodes of this tree represent various blending operations that are being performed on the clips. Multiple blend operations are composed to form action states. Additional blend nodes are introduced to represent transient cross-fades. And in a layered ASM, the output poses obtained from the action states in each layer are blended together. The final pose of the character is thus produced at the root of this potentially complex blend tree.

12.10.1 The Flat Weighted Average Approach

In the flat weighted average approach, every animation clip that is currently playing on a given character is associated with a blend weight indicating how much it should contribute to its final pose. A flat list of all *active* animation clips (i.e., clips whose blend weights are nonzero) is maintained. To calculate the final pose of the skeleton, we extract a pose at the appropriate time index for each of the N active clips. Then, for each joint of the skeleton, we calculate a simple N-point weighted average of the translation vectors, rotation quaternions and scale factors extracted from the N active animations. This yields the final pose of the skeleton.

The equation for the weighted average of a set of N vectors $\{\mathbf{v}_i\}$ is as follows:

$$\mathbf{v}_{\text{avg}} = \frac{\sum_{i=0}^{N-1} w_i \mathbf{v}_i}{\sum_{i=0}^{N-1} w_i}.$$

If the weights are *normalized*, meaning they sum to one, then this equation can be simplified to the following:

$$\mathbf{v}_{\text{avg}} = \sum_{i=0}^{N-1} w_i \mathbf{v}_i, \qquad \text{when } \sum_{i=0}^{N-1} w_i = 1.$$

In the case of $N = 2$, if we let $w_0 = (1 - \beta)$ and $w_1 = \beta$, the weighted average reduces to the familiar equation for the linear interpolation (LERP) between two vectors:

$$\begin{aligned}\mathbf{v}_{\text{avg}} &= w_0\mathbf{v}_A + w_1\mathbf{v}_B \\ &= (1-\beta)\,\mathbf{v}_A + \beta\mathbf{v}_B \\ &= \text{LERP}\left[\mathbf{v}_A, \mathbf{v}_B, \beta\right].\end{aligned}$$

We can apply this same weighted average formulation equally well to quaternions by simply treating them as four-element vectors.

12.10.1.1 Example: OGRE

The OGRE animation system works in exactly this way. An `Ogre::Entity` represents an instance of a 3D mesh (e.g., one particular character walking around in the game world). The `Entity` aggregates an object called an `Ogre::AnimationStateSet`, which in turn maintains a list of `Ogre::AnimationState` objects, one for each active animation. The `Ogre::AnimationState` class is shown in the code snippet below. (A few irrelevant details have been omitted for clarity.)

```
/** Represents the state of an animation clip and the
    weight of its influence on the overall pose of the
    character.
*/
class AnimationState
{
protected:
    String              mAnimationName; // reference to
                                        // clip
    Real                mTimePos;       // local clock
    Real                mWeight;        // blend weight
    bool                mEnabled;       // is this anim
                                        // running?
    bool                mLoop;          // should the
                                        // anim loop?

public:
    /// API functions...
};
```

Each `AnimationState` keeps track of one animation clip's local clock and its blend weight. When calculating the final pose of the skeleton for a particular `Ogre::Entity`, OGRE's animation system simply loops through each

active `AnimationState` in its `AnimationStateSet`. A skeletal pose is extracted from the animation clip corresponding to each state at the time index specified by that state's local clock. For each joint in the skeleton, an N-point weighted average is then calculated for the translation vectors, rotation quaternions and scales, yielding the final skeletal pose.

It is interesting to note that OGRE has no concept of a playback rate (R). If it did, we would have expected to see a data member like this in the `Ogre::AnimationState` class:

```
Real    mPlaybackRate;
```

Of course, we can still make animations play more slowly or more quickly in OGRE by simply scaling the amount of time we pass to the `addTime()` function, but unfortunately, OGRE does not support animation time scaling out of the box.

12.10.1.2 Example: Granny

The Granny animation system, by Rad Game Tools (http://www.radgametool.com/granny.html), provides a flat, weighted average animation blending system similar to OGRE's. Granny permits any number of animations to be played on a single character simultaneously. The state of each active animation is maintained in a data structure known as a `granny_control`. Granny calculates a weighted average to determine the final pose, automatically normalizing the weights of all active clips. In this sense, its architecture is virtually identical to that of OGRE's animation system.

Where Granny really shines is in its handling of time. Granny uses the global clock approach discussed in Section 12.4.3. It allows each clip to be looped an arbitrary number of times or infinitely. Clips can also be time-scaled; a negative time scale allows an animation to be played in reverse.

12.10.1.3 Cross-Fades with a Flat Weighted Average

In an animation engine that employs the flat weighted average architecture, cross-fades are implemented by adjusting the weights of the clips themselves. Recall that any clip whose weight $w_i = 0$ will not contribute to the current pose of the character, while those whose weights are nonzero are averaged together to generate the final pose. If we wish to transition smoothly from clip A to clip B, we simply ramp up clip B's weight w_B, while simultaneously ramping down clip A's weight w_A. This is illustrated in Figure 12.49.

Cross-fading in a weighted average architecture becomes a bit trickier when we wish to transition from one complex blend to another. As an example, let's say we wish to transition the character from walking to jumping.

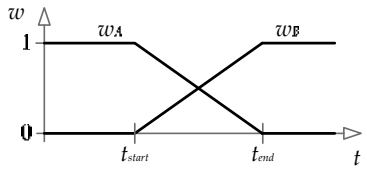

Figure 12.49. A simple cross-fade from clip A to clip B, as implemented in a weighted average animation architecture.

Let's assume that the walk movement is produced by a three-way average between clips A, B and C, and that the jump movement is produced by a two-way average between clips D and E.

We want the character to look like he's smoothly transitioning from walking to jumping, without affecting how the walk or jump animations look individually. So during the transition, we want to ramp down the ABC clips and ramp up the DE clips while keeping the *relative weights* of the ABC and DE clip groups constant. If the cross-fade's blend factor is denoted by λ, we can meet this requirement by simply setting the weights of *both* clip groups to their desired values and then multiplying the weights of the source group by $(1-\lambda)$ and the weights of the destination group by λ.

Let's look at a concrete example to convince ourselves that this will work properly. Imagine that before the transition from ABC to DE, the nonzero weights are as follows: $w_A = 0.2$, $w_B = 0.3$ and $w_C = 0.5$. After the transition, we want the nonzero weights to be $w_D = 0.33$ and $w_E = 0.66$. So, we set the weights as follows:

$$\begin{aligned} w_A &= (1-\lambda)(0.2), & w_D &= \lambda(0.33), \\ w_B &= (1-\lambda)(0.3), & w_E &= \lambda(0.66). \\ w_C &= (1-\lambda)(0.5), \end{aligned} \tag{12.20}$$

From Equations (12.20), you should be able to convince yourself of the following:

1. When $\lambda = 0$, the output pose is the correct blend of clips A, B and C, with zero contribution from clips D and E.

2. When $\lambda = 1$, the output pose is the correct blend of clips D and E, with no contribution from A, B or C.

3. When $0 < \lambda < 1$, the *relative* weights of both the ABC group and the DE group remain correct, although they no longer add to one. (In fact, group ABC's weights add to $(1-\lambda)$, and group DE's weights add to λ.)

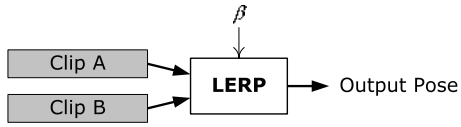

Figure 12.50. A binary LERP blend, represented by a binary expression tree.

For this approach to work, the implementation must keep track of the logical groupings between clips (even though, at the lowest level, all of the clips' states are maintained in one big, flat array—for example, the `Ogre::AnimationStateSet` in OGRE). In our example above, the system must "know" that A, B and C form a group, that D and E form another group, and that we wish to transition from group ABC to group DE. This requires additional metadata to be maintained, on top of the flat array of clip states.

12.10.2 Blend Trees

Some animation engines represent a character's clip state not as a flat weighted average but rather as a tree of blend operations. An animation blend tree is an example of what is known in compiler theory as an *expression tree* or a *syntax tree*. The interior nodes of such a tree are operators, and the leaf nodes serve as the inputs to those operators. (More correctly, the interior nodes represent the *nonterminals* of the grammar, while the leaf nodes represent the *terminals*.)

In the following sections, we'll briefly revisit the various kinds of animation blends we learned about in Sections 12.6.3 and 12.6.5 and see how each can be represented by an expression tree.

12.10.2.1 Binary LERP Blend Trees

As we saw in Section 12.6.1, a binary linear interpolation (LERP) blend takes two input poses and blends them together into a single output pose. A blend weight β controls the percentage of the second input pose that should appear at the output, while $(1 - \beta)$ specifies the percentage of the first input pose. This can be represented by the binary expression tree shown in Figure 12.50.

12.10.2.2 Generalized One-Dimensional Blend Trees

In Section 12.6.3.1, we learned that it can be convenient to define a generalized one-dimensional LERP blend by placing an arbitrary number of clips along a linear scale. A blend factor b specifies the desired blend along this scale. Such a blend can be pictured as an n-input operator, as shown in Figure 12.51.

Given a specific value for b, such a linear blend can always be transformed into a binary LERP blend. We simply use the two clips immediately adjacent to

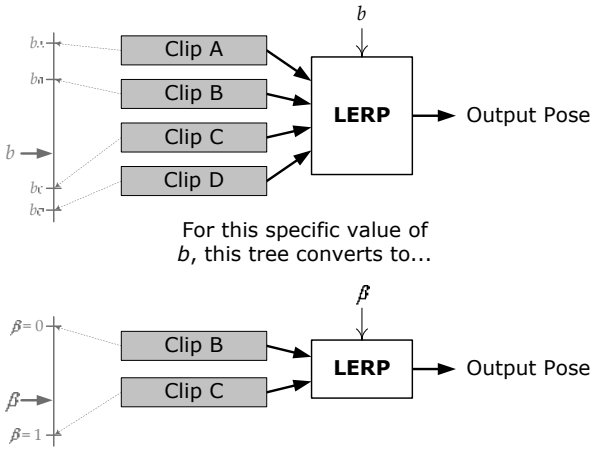

Figure 12.51. A multi-input expression tree can be used to represent a generalized 1D blend. Such a tree can always be transformed into a binary expression tree for any specific value of the blend factor b.

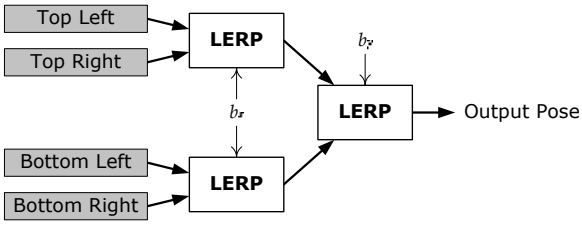

Figure 12.52. A simple 2D LERP blend, implemented as cascaded binary blends.

b as the inputs to the binary blend and calculate the blend weight β as specified in Equation (12.15)

12.10.2.3 Two-Dimensional LERP Blend Trees

In Section 12.6.3.2, we saw how a two-dimensional LERP blend can be realized by simply cascading the results of two binary LERP blends. Given a desired two-dimensional blend point $\mathbf{b} = \begin{bmatrix} b_x & b_y \end{bmatrix}$, Figure 12.52 shows how this kind of blend can be represented in tree form.

12.10.2.4 Additive Blend Trees

Section 12.6.5 described additive blending. This is a binary operation, so it can be represented by a binary tree node, as shown in Figure 12.53. A single blend weight β controls the amount of the additive animation that should appear at

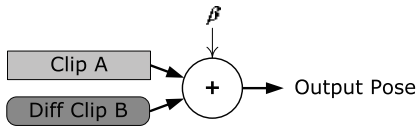

Figure 12.53. An additive blend represented as a binary tree.

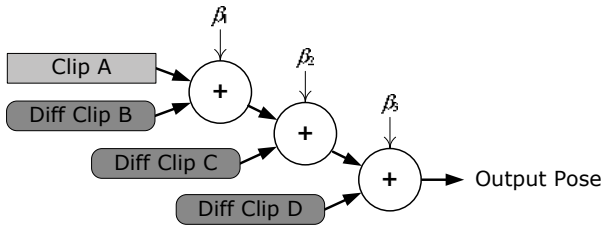

Figure 12.54. In order to additively blend more than one difference pose onto a regular "base" pose, a cascaded binary expression tree must be used.

the output—when $\beta = 0$, the additive clip does not affect the output at all, while when $\beta = 1$, the additive clip has its maximum effect on the output.

Additive blend nodes must be handled carefully, because the inputs are not interchangeable (as they are with most types of blend operators). One of the two inputs is a regular skeletal pose, while the other is a special kind of pose known as a *difference pose* (also known as an *additive pose*). A difference pose may *only* be applied to a regular pose, and the result of an additive blend is another regular pose. This implies that the additive input of a blend node must always be a leaf node, while the regular input may be a leaf or an interior node. If we want to apply more than one additive animation to our character, we must use a cascaded binary tree with the additive clips always applied to the additive inputs, as shown in Figure 12.54.

12.10.2.5 Layered Blend Trees

We said at the beginning of Section 12.10 that complex character movement can be produced by arranging multiple independent state machines into *state layers*. The output poses from each layer's ASM are blended together into a final composite pose. When this is implemented using blend trees, the net effect is to combine the blend trees of each active state together into one über tree, as illustrated in Figure 12.55.

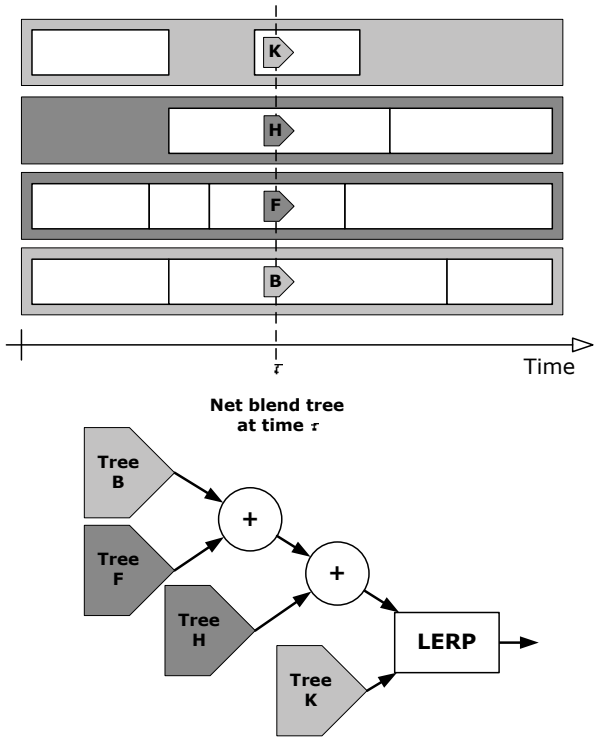

Figure 12.55. A layered state machine converts the blend trees from multiple states into a single, unified tree.

12.10.2.6 Cross-Fades with Blend Trees

As a character transitions from state to state within each layer of a layered ASM, we often wish to provide a smooth cross-fade between states. Implementing a cross-fade in an expression tree based ASM is a bit more intuitive than it is in a weighted average architecture. Whether we're transitioning from one clip to another or from one complex blend to another, the approach is always the same: We simply introduce a transient binary LERP node between the roots of the blend trees of each state to handle the cross-fade.

We'll denote the blend factor of the cross-fade node with the symbol λ as before. Its top input is the source state's blend tree (which can be a single clip or a complex blend), and its bottom input is the destination state's tree (again a clip or a complex blend). During the transition, λ is ramped from zero to one. Once $\lambda = 1$, the transition is complete, and the cross-fade LERP node and its top input tree can be retired. This leaves its bottom input tree as the root of

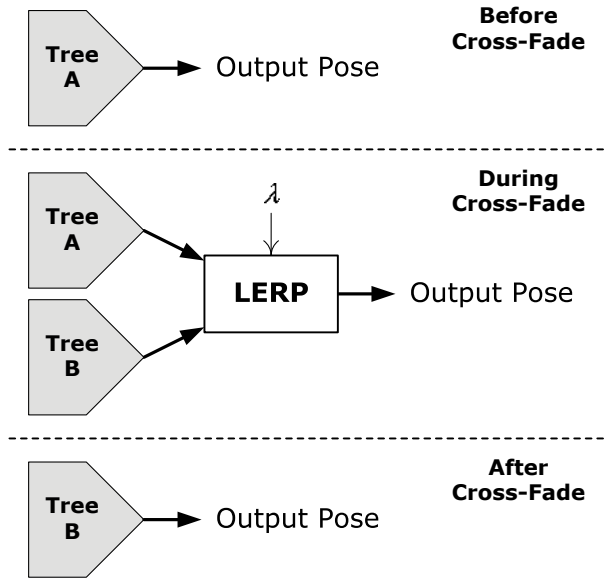

Figure 12.56. A cross-fade between two arbitrary blend trees A and B.

the overall blend tree for the given state layer, thus completing the transition. This process is illustrated in Figure 12.56.

12.10.3 State and Blend Tree Specifications

Animators, game designers and programmers usually cooperate to create the animation and control systems for the central characters in a game. These developers need a way to specify the states that make up a character's ASM, to lay out the tree structure of each blend tree, and to select the clips that will serve as their inputs. Although the states and blend trees could be hard-coded, most modern game engines provide a *data-driven* means of defining animation states. The goal of a data-driven approach is to permit a user to create new animation states, remove unwanted states, fine-tune existing states and then see the effects of his or her changes reasonably quickly. In other words, the central goal of a data-driven animation engine is to enable *rapid iteration*.

To build an arbitrarily complex blend tree, we really only require four atomic types of blend nodes: clips, binary LERP blends, binary additive blends and possibly ternary (triangular) LERP blends. Virtually any blend tree imaginable can be created as compositions of these atomic nodes.

A blend tree built exclusively from atomic nodes can quickly become large and unwieldy. As a result, many game engines permit custom compound node types to be predefined for convenience. The *N*-dimensional linear blend node discussed in Sections 12.6.3.4 and 12.10.2.2 is an example of a compound node. One can imagine myriad complex blend node types, each one addressing a particular problem specific to the particular game being made. A soccer game might define a node that allows the character to dribble the ball. A war game could define a special node that handles aiming and firing a weapon. A brawler could define custom nodes for each fight move the characters can perform. Once we have the ability to define custom node types, the sky's the limit.

The means by which the users enter animation state data varies widely. Some game engines employ a simple, bare-bones approach, allowing animation states to be specified in a text file with a simple syntax. Other engines provide a slick, graphical editor that permits animation states to be constructed by dragging atomic components such as clips and blend nodes onto a canvas and linking them together in arbitrary ways. Such editors usually provide a live preview of the character so that the user can see immediately how the character will look in the final game. In my opinion, the specific method chosen has little bearing on the quality of the final game—what matters most is that the user can make changes and see the results of those changes reasonably quickly and easily.

12.10.3.1 Example: The Naughty Dog Engine

The animation engine used in Naughty Dog's *Uncharted* and *The Last of Us* franchises employs a simple, text-based approach to specifying animation states. For reasons related to Naughty Dog's rich history with the Lisp language (see Section 16.9.5.1), state specifications in the Naughty Dog engine are written in a customized version of the Scheme programming language (which itself is a Lisp variant). Two basic state types can be used: *simple* and *complex*.

Simple States

A *simple* state contains a single animation clip. For example:

```
(define-state simple
    :name "pirate-b-bump-back"
    :clip "pirate-b-bump-back"
    :flags (anim-state-flag no-adjust-to-ground)
)
```

Don't let the Lisp-style syntax throw you. All this block of code does is to define a state named "pirate-b-bump-back" whose animation clip also happens to be named "pirate-b-bump-back." The `:flags` parameter allows users to specify various Boolean options on the state.

Complex States

A *complex* state contains an arbitrary tree of LERP or additive blends. For example, the following state defines a tree that contains a single binary LERP blend node, with two clips ("walk-l-to-r" and "run-l-to-r") as its inputs:

```
(define-state complex
    :name "move-l-to-r"
    :tree
        (anim-node-lerp
            (anim-node-clip "walk-l-to-r")
            (anim-node-clip "run-l-to-r")
        )
)
```

The `:tree` argument allows the user to specify an arbitrary blend tree, composed of LERP or additive blend nodes and nodes that play individual animation clips.

From this, we can see how the `(define-state simple ...)` example shown above might really work under the hood—it probably defines a complex blend tree containing a single "clip" node, like this:

```
(define-state complex
    :name "pirate-b-unimog-bump-back"
    :tree (anim-node-clip "pirate-b-unimog-bump-back")
    :flags (anim-state-flag no-adjust-to-ground)
)
```

The following complex state shows how blend nodes can be cascaded into arbitrarily deep blend trees:

```
(define-state complex
    :name "move-b-to-f"
    :tree
        (anim-node-lerp
            (anim-node-additive
                (anim-node-additive
                    (anim-node-clip "move-f")
                    (anim-node-clip "move-f-look-lr")
                )
```

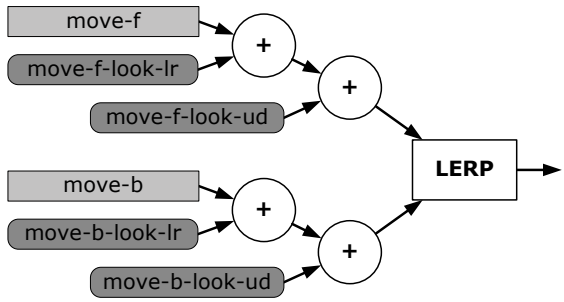

Figure 12.57. Blend tree corresponding to the example state "move-b-to-f."

```
                (anim-node-clip "move-f-look-ud")
            )
            (anim-node-additive
                (anim-node-additive
                    (anim-node-clip "move-b")
                    (anim-node-clip "move-b-look-lr")
                )
                (anim-node-clip "move-b-look-ud")
            )
        )
    )
```

This corresponds to the tree shown in Figure 12.57.

Rapid Iteration

Naughty Dog's animation team achieves rapid iteration with the help of four important tools:

1. An in-game animation viewer allows a character to be spawned into the game and its animations controlled via an in-game menu.
2. A simple command-line tool allows animation scripts to be recompiled and reloaded into the running game on the fly. To tweak a character's animations, the user can make changes to the text file containing the animation state specifications, quickly reload the animation states and immediately see the effects of his or her changes on an animating character in the game.
3. The engine continually keeps track of all state transitions performed by each character during the last few seconds of gameplay. This allows us to pause the game and then literally rewind the animations to scrutinize them and debug problems that are noticed while playing.

4. The Naughty Dog engine also offers a host of "live update" tools. For example, animators can tweak their animations in Maya and see them update virtually instantaneously in the game.

12.10.3.2 Example: Unreal Engine 4

Unreal Engine 4 (UE4) provides its users with five tools for working with skeletal animations and skeletal meshes: The Skeleton Editor, the Skeletal Mesh Editor, the Animation Editor, the Animation Blueprint Editor, and the Physics Editor.

- The Skeleton Editor is essentially a rigging tool. It allows users to view and modify skeletons, add *sockets* to joints, and test out the movement of the skeleton. A socket is sometimes called an *attach point* in other engines (see Section 12.11.1).
- The Skeletal Mesh Editor allows users to edit properties of the meshes that are skinned to animating skeletons.
- The Animation Editor allows users to import, create and manage animation assets. In this editor, the compression and timing of animation clips (which UE4 calls Sequences) can be adjusted. Clips can be combined into predefined Blend Spaces, and in-game cinematics can be defined by creating Animation Montages.
- The Animation Blueprint Editor allows users to apply the power of Unreal Engine's Blueprints visual scripting system to controlling characters' animation state machines. This editor is depicted in Figure 12.58.
- The Physics Editor allows users to model a hierarchy of rigid bodies that drive the skeleton's motion when ragdoll physics is active.

A complete discussion of Unreal Engine's animation tools is beyond our scope here, but you can read more about it by searching for "Unreal Skeletal Mesh Animation System" online.

12.10.4 Transitions

To create a high-quality animating character, we must carefully manage the *transitions* between states in the action state machine to ensure that the splices between animations do not have a jarring and unpolished appearance. Most modern animation engines provide a data-driven mechanism for specifying exactly how transitions should be handled. In this section, we'll explore how this mechanism works.

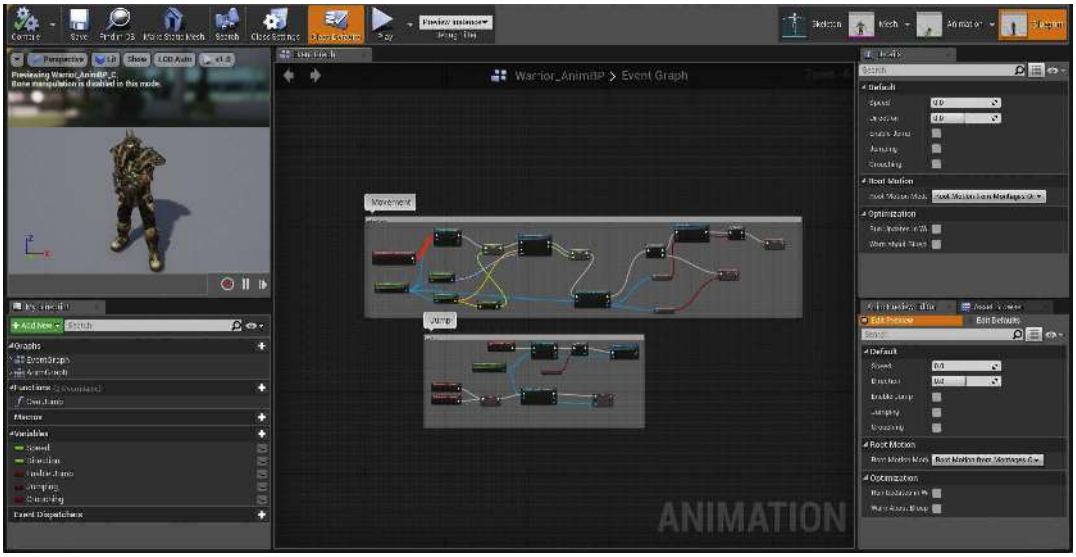

Figure 12.58. The Unreal Engine 4 animation Blueprints editor. (See Color Plate XXVI.)

12.10.4.1 Kinds of Transitions

There are many different ways to manage the transition between states. If we know that the final pose of the source state exactly matches the first pose of the destination state, we can simply "pop" from one state to another. Otherwise, we can cross-fade from one state to the next. Cross-fading is not always a suitable choice when transitioning from state to state. For example, there is no way that a cross-fade can produce a realistic transition from lying on the ground to standing upright. For this kind of state transition, we need one or more custom animations. This kind of transition is often implemented by introducing special *transitional states* into the state machine. These states are intended for use only when going from one state to another—they are never used as a steady-state node. But because they are full-fledged states, they can be comprised of arbitrarily complex blend trees. This provides maximum flexibility when authoring custom-animated transitions.

12.10.4.2 Transition Parameters

When describing a particular transition between two states, we generally need to specify various parameters, controlling exactly how the transition will occur. These include but are not limited to the following.

- *Source and destination states*. To which state(s) does this transition apply?

- *Transition type*. Is the transition immediate, cross-faded or performed via a transitional state?
- *Duration*. For cross-faded transitions, we need to specify how long the cross-fade should take.
- *Ease-in/ease-out curve type*. In a cross-faded transition, we may wish to specify the type of ease-in/ease-out curve to use to vary the blend factor during the fade.
- *Transition window*. Certain transitions can only be taken when the source animation is within a specified window of its local timeline. For example, a transition from a punch animation to an impact reaction might only make sense when the arm is in the second half of its swing. If an attempt to perform the transition is made during the first half of the swing, the transition would be disallowed (or a different transition might be selected instead).

12.10.4.3 The Transition Matrix

Specifying transitions between states can be challenging, because the number of possible transitions is usually very large. In a state machine with n states, the worst-case number of possible transitions is n^2. We can imagine a two-dimensional square matrix with every possible state listed along both the vertical and horizontal axes. Such a table can be used to specify all of the possible transitions from any state along the vertical axis to any other state along the horizontal axis.

In a real game, this *transition matrix* is usually quite sparse, because not all state-to-state transitions are possible. For example, transitions are usually disallowed from a death state to any other state. Likewise, there is probably no way to go from a driving state to a swimming state (without going through at least one intermediate state that causes the character to jump out of his vehicle). The number of unique transitions in the table may be significantly less even than the number of valid transitions between states. This is because we can often reuse a single transition specification between many different pairs of states.

12.10.4.4 Implementing a Transition Matrix

There are all sorts of ways to implement a transition matrix. We could use a spreadsheet application to tabulate all the transitions in matrix form, or we might permit transitions to be authored in the same text file used to author our action states. If a graphical user interface is provided for state editing, transitions could be added to this GUI as well. In the following sections, we'll

take a brief look at a few transition matrix implementations from real game engines.

Example: Wildcarded Transitions in Medal of Honor: Pacific Assault

On *Medal of Honor: Pacific Assault* (*MOHPA*), we used the sparseness of the transition matrix to our advantage by supporting wildcarded transition specifications. For each transition specification, the names of both the source and destination states could contain asterisks (*) as a wildcard character. This allowed us to specify a single default transition from any state to any other state (via the syntax `from="*" to="*"`) and then refine this global default easily for entire categories of states. The refinement could be taken all the way down to custom transitions between specific state pairs when necessary. The *MOHPA* transition matrix looked something like this:

```
<transitions>
    <!-- global default -->
    <trans from="*" to="*"
           type=frozen duration=0.2>

    <!-- default for any walk to any run -->
    <trans from="walk*" to="run*"
           type=smooth
           duration=0.15>

    <!-- special handling from any prone to any getting-up
      -- action (only valid from 2 sec to 7.5 sec on the
      -- local timeline) -->
    <trans from="*prone" to="*get-up"
           type=smooth
           duration=0.1
           window-start=2.0
           window-end=7.5>
    ...
</transitions>
```

Example: First-Class Transitions in Uncharted

In some animation engines, high-level game code requests transitions from the current state to a new state by naming the destination state explicitly. The problem with this approach is that the calling code must have intimate knowledge of the names of the states and of which transitions are valid when in a particular state.

In Naughty Dog's engine, this problem is overcome by turning state transitions from secondary implementation details into first-class entities. Each state provides a list of valid transitions to other states, and each transition is given a unique name. The names of the transitions are standardized in order to make the *effect* of each transition predictable. For example, if a transition is called "walk," then it *always* goes from the current state to a walking state of some kind, no matter what the current state is. Whenever the high-level animation control code wants to transition from state A to state B, it asks for a transition by name (rather than requesting the destination state explicitly). If such a transition can be found and is valid, it is taken; otherwise, the request fails.

The following example state defines four transitions named "reload," "step-left," "step-right" and "fire." The `(transition-group ...)` line invokes a previously defined group of transitions; it is useful when the same set of transitions is to be used in multiple states. The `(transition-end ...)` command specifies a transition that is taken upon reaching the end of the state's local timeline if no other transition has been taken before then.

```
(define-state complex
    :name "s_turret-idle"
    :tree (aim-tree
              (anim-node-clip "turret-aim-all--base")
              "turret-aim-all--left-right"
              "turret-aim-all--up-down"
          )
    :transitions (
        (transition "reload" "s_turret-reload"
            (range - -) :fade-time 0.2)

        (transition "step-left" "s_turret-step-left"
            (range - -) :fade-time 0.2)

        (transition "step-right" "s_turret-step-right"
            (range - -) :fade-time 0.2)

        (transition "fire" "s_turret-fire"
            (range - -) :fade-time 0.1)

        (transition-group "combat-gunout-idle^move")

        (transition-end "s_turret-idle")
    )
)
```

The beauty of this approach may be difficult to see at first. Its primary

purpose is to allow transitions and states to be modified in a data-driven manner, without requiring changes to the C++ source code in many cases. This degree of flexibility is accomplished by shielding the animation control code from knowledge of the structure of the state graph. For example, let's say that we have ten different walking states (normal, scared, crouched, injured and so on). All of them can transition into a jumping state, but different kinds of walks might require different jump animations (e.g., normal jump, scared jump, jump from crouch, injured jump, etc.). For each of the ten walking states, we define a transition simply called "jump." At first, we can point all of these transitions to a single generic "jump" state, just to get things up and running. Later, we can fine-tune some of these transitions so that they point to custom jump states. We can even introduce transitional states between some of the "walk" states and their corresponding "jump" states. All sorts of changes can be made to the structure of the state graph and the parameters of the transitions without affecting the C++ source code—as long as the *names* of the transitions don't change.

12.10.5 Control Parameters

From a software engineering perspective, it can be challenging to orchestrate all of the blend weights, playback rates and other control parameters of a complex animating character. Different blend weights have different effects on the way the character animates. For example, one weight might control the character's movement direction, while others control its movement speed, horizontal and vertical weapon aim, head/eye look direction and so on. We need some way of exposing all of these blend weights to the code that is responsible for controlling them.

In a flat weighted average architecture, we have a flat list of all the animation clips that could possibly be played on the character. Each clip state has a blend weight, a playback rate and possibly other control parameters. The code that controls the character must look up individual clip states by name and adjust each one's blend weight appropriately. This makes for a simple interface, but it shifts most of the responsibility for controlling the blend weights to the character control system. For example, to adjust the direction in which a character is running, the character control code must know that the "run" action is comprised of a group of animation clips, named something like "StrafeLeft," "RunForward," "StrafeRight" and "RunBackward." It must look up these clip states by name and manually control all four blend weights in order to achieve a particular angled run animation. Needless to say, controlling animation parameters in such a fine-grained way can be tedious and can lead to difficult-